

1.整体介绍:

基于BCC开发，内核层使用eBPF, 用户态使用python进行开发。

功能:

- 1.定时收集当前系统环境中物理内存碎片化程度信息，定时时间可配置；
- 2.记录内存碎片化函数数据信息；
- 3.使用一种可视化工具对当前收集的物理内存碎片化信息进行直观展示

代码架构介绍:

- `extfrag.py` 文件，用于实现提取相应格式的数据函数
- `extfrag_user.py` 文件，用于实现命令行接口
- `extfraginfo.c` 实现监测外碎片化事件
- `fraginfo.c` 统计系统中所有内存节点中的所有 `zone` 对于不同 `order` 的碎片化程度

采集的碎片化信息如下:

采集的碎片化程度信息如下:

- ZONE_COMM : 表示 `zone` 的名称，有DMA/NORMAL/DMA32等
- ZONE_PFN : 表示该内存区域从哪一个物理页框号开始。
- SUM_PAGES : 此区域内的总页数，指内存区域总共包含的物理内存页数。
- FACT_PAGES : 此区域实际使用中的页数
- ORDER : 表示页块的大小
- TOTAL : 该区域内空闲块的总数
- SUITABLE : 适合当前分配请求的空闲块数
- FREE: 该区域内空闲页的总数
- NODE_ID : 表示内存节点的标识符
- extfrag_index : 表示内核中 `extfrag_index` 指数
- unusable_index : 表示内核中的 `unusable_index` 指数

采集的节点信息如下:

- NODE_ID: 表示内存节点的标识符
- Number of Zones: 节点中的区域个数
- NODE_START_PFN: 节点的 `pgdat` 的起始页帧号

采集的外碎片化事件信息如下:

- COMM: 发生外碎片化事件的进程名
- PID: 发生外碎片化事件的进程号
- PFN: 表示实际分配的物理页的页帧号
- ALLOC_ORDER: 初始分配内存的阶数
- FALLBACK_ORDER: 在分配请求无法满足时，实际分配到的内存块的阶数
- COUNT: 发生外碎片化事件的次数

1.比如第一个**eBPF程序就是**: `extfraginfo.c` : `ebpf`代码，监控内存碎片化分配: 通过

`TRACEPOINT_PROBE(kmem, mm_page_alloc_extfrag)`挂钩内核的`mm_page_alloc_extfrag`事件，该事件在内存分配因碎片化需要降级 (fallback) 时触发。

BPF_HASH的方式和用户态共享数据，还用了一个BPF_map记录延迟时间，存储上一次运行的时间，在通过**bpf_ktime_get_ns()**获取当前时间进行差值比较看是否继续收集；tracepoint_probe他挂载的这个静态点的args参数信息，

- **数据记录**：捕获每次碎片化分配的详细信息，包括：

- **物理页帧号 (PFN)**：args->pfn，标识分配的物理内存位置。
- **分配阶数**：args->**alloc_order**（请求的连续页块阶数）和args->**fallback_order**（实际分配的阶数，可能更小）。（可能因为不足而降级）
- **进程信息**：进程PID和名称（pcomm），用于关联分配行为到具体进程。（通过辅助函数获得，
`bpf_get_current_comm(&data->pcomm, sizeof(data->pcomm));`）
- 最后就是通过 `counts_map.update(&pid, data);` **更新数据到hash共享**；

2.fraginfo.c ebpf代码：

eBPF kprobe的触发与回调机制

- **动态插桩**：通过kprobe机制，eBPF程序将kprobe__get_page_from_freelist函数挂钩到get_page_from_freelist的入口，每次调用get_page_from_freelist时，会先执行该回调函数。
(get_page_from_freelist 是 Linux 内核伙伴系统 (Buddy System) 的核心函数，负责在快速路径 (fast path) 中尝试从空闲内存链表中分配物理页面)
- **回调条件**：
 - **时间过滤**：通过last_time_map和delay_map控制采样频率，避免高频触发影响性能。
 - **参数传递**：回调函数通过struct pt_regs *ctx和函数签名（如alloc_context *ac）获取内核函数的原始参数进行获取信息：

数据记录 pgdat_info结构体：节点初始页帧号，包含的内存区域zone数量，numa节点ID

zone_info结构体：区域起始页帧号，实际可用页数，总空闲页数，总空闲块数，满足当前阶数的连续块数，统计当前阶数，碎片化指数（a），不可用内存比例（b），所属NUMA节点ID
for循环便利所有zonelist，然后看他属于哪个numa，然后将此numa数据放到pgdat_info结构体中。如果此节点位在pgdat_map中记录，则初始化。

eBPF 程序的加载与内核交互流程

- **加载阶段**：当执行 `sudo ./extfrag_user.py -n` 时，`extfrag.py` 中的 `self.b = BPF(src_file="./bpf/fraginfo.c")` 会完成以下操作：
 - a. **编译与加载**：BCC 将 `fraginfo.c` 编译为 eBPF 字节码，并通过 `bpf()` 系统调用将其加载到内核。
此时 eBPF 程序已挂载到目标内核函数
(如 `get_page_from_freelist` 或 `mm_page_alloc_extfrag`) 的探针点 (kprobe/kretprobe)。
 - b. **参数传递**：`self.b["delay_map"][delay_key] = ctypes.c_int(interval)` 通过 eBPF 映射 (map) 向内核传递参数 (如采样间隔 `interval`)，控制 eBPF 程序的行为。
- **数据收集阶段**：eBPF 程序**不会主动触发执行**，而是**被动等待内核函数被调用**。例如：
 - 当内核执行 `get_page_from_freelist` 时，挂载的 eBPF 探针被触发，收集内存分配信息 (如 `zone` 数据、碎片化指标等)。
 - 收集的数据通过 eBPF 映射 (如 `perf_event_array` 或哈希表) 实时传递到用户态。

2.项目的必要性:

背景:

内存碎片化是指物理内存中的空闲区域无法满足某些大内存请求的需求。内存碎片化可能导致系统无法为某些进程分配连续的内存页，从而使得本可以利用的内存空间无法有效使用。随着内存的不断分配和释放，这种碎片化现象会逐渐加重，尤其是在没有足够空间来满足大内存请求时，可能会导致系统性能下降，甚至发生内存分配失败。

在 Linux 系统中，内核使用了伙伴系统 (Buddy System) 来进行内存分配，但这个系统并不能完全解决内存碎片化的问题。碎片化会导致内存分配不均匀，导致性能下降，甚至引发内存不足错误。在某些情况下，即便系统有足够的物理内存，由于碎片化的影响，系统仍然无法满足分配大块连续内存的需求。

因此，必须有一种方式来监控、检测和评估内存碎片化的程度，及时采取措施，如内存压缩 (compaction) 或其他内存优化机制，从而提高系统的内存利用效率和稳定性。

用途与解决的问题:

1. 监控内存碎片化情况:

- 功能: 通过基于 eBPF 的解决方案，能够实时监控和收集内存碎片化的详细信息，如内存区域的空闲块情况、碎片化指数、实际分配的内存块大小等。这些信息能够帮助开发人员或系统管理员实时了解系统内存的碎片化程度。
- 问题: 系统在运行过程中，可能无法实时掌握内存碎片化的详细信息。只有通过定时收集和精确的监控，才能更准确地判断碎片化是否已经影响到内存的正常分配。

2. 数据采集与分析:

- 功能: 系统通过 eBPF 程序 (如 `extfraginfo.c` 和 `fraginfo.c`) 精确记录内存碎片化的各项指标，例如物理内存的使用情况、空闲块的分布、每个 NUMA 节点的内存碎片化情况等。这些数据可以被用户态程序 (如 Python 脚本) 处理和可视化，以便进行详细分析。
- 问题: 没有足够细粒度的内存碎片化信息时，可能很难确定碎片化的具体表现或是产生性能问题的原因。这种方法可以帮助定位问题来源。

3. 内存碎片化诊断与优化:

- 功能: 通过监测 `mm_page_alloc_extfrag` 和 `get_page_from_freelist` 等内核事件，当内存分配因碎片化无法满足请求时，能够实时获取相关数据，帮助系统及时诊断碎片化问题。通过这些数据，可以触发内存规整 (compaction) 等机制来优化内存分配。
- 问题: 内核默认并不会提供足够的内存碎片化诊断信息，这会让系统管理员或开发人员很难明确知道是否是内存碎片化造成了分配失败或性能下降。这套系统能够精准捕捉碎片化事件，并通过收集的数据判断何时需要规整内存。

4. 提高系统性能与稳定性:

- 功能: 通过定期分析碎片化情况，能够在合适的时机采取措施 (如内存压缩、调整分配策略等)，减少碎片化对内存分配的影响，从而提升系统的内存利用率和性能。
- 问题: 系统长时间运行后，内存碎片化可能导致某些大内存请求无法成功分配，进而影响应用程序的运行性能。实时监控碎片化情况，并采取适当的优化措施，可以显著减少这种影响，保持系统稳定。

5. 可视化与反馈：

- 功能：通过收集的数据生成可视化展示，帮助系统管理员更直观地理解当前内存碎片化的状态。例如，展示不同内存区域（如 DMA、NORMAL 等）内的碎片化程度和空闲块信息。
- 问题：如果没有直观的可视化工具，系统管理员很难及时了解内存的具体状态。这种可视化工具可以帮助他们快速做出决策，如是否需要进行内存压缩，或者是否需要优化内存分配策略。

总结：

这个基于 eBPF 的内存碎片化监测系统能够解决以下问题：

1. 实时监控内存碎片化情况：提供细粒度的内存碎片化数据，帮助开发人员和管理员随时掌握内存碎片化的状态。
2. 数据采集与分析：通过精确的数据采集和分析，帮助快速定位碎片化问题。
3. 优化内存分配：通过捕获外碎片化事件并分析数据，优化内存分配策略，提高系统性能。
4. 提高系统稳定性：及时发现和解决碎片化问题，避免由于碎片化导致的内存分配失败或性能下降。
5. 可视化反馈：通过直观的可视化工具帮助管理员理解内存碎片化情况，从而更好地进行优化决策。

3.fraginfo.c:

节点信息 (`pgdat_info`)

```
1 struct pgdat_info {
2     u64 pgdat_ptr;           // NUMA节点指针
3     int nr_zones;           // 节点包含的zone数量
4     int node_id;            // NUMA节点ID
5 };
```

- **用途：**记录NUMA节点的基本信息，用于分析多节点内存分配行为。

内存区域信息 (`zone_info`)

```
1 struct zone_info {
2     u64 zone_ptr;           // zone结构体指针
3     u64 zone_start_pfn;     // 起始页帧号
4     u64 spanned_pages;      // zone总页数（含空洞）
5     u64 present_pages;      // 实际存在的页数
6     unsigned long free_pages; // 空闲页总数
```

```

7  unsigned long free_blocks_total;    // 空闲块总数（所有order）
8  unsigned long free_blocks_suitable; // 适合当前请求order的块数，大于所需的order的块数
9  char name[32];                    // zone名称（如"DMA"、"Normal"）
10 int order;                         // 当前分析的分配阶数
11 int score_a;                       // 碎片化指数A（标准碎片化指数）
12 int score_b;                       // 碎片化指数B（不可用页指数）
13 int node_id;                       // 所属NUMA节点ID
14 };

```

- **用途：**记录每个内存区域（zone）的详细状态，包括空闲内存和碎片化指标。
- **块 (Block) 定义：**Linux 伙伴系统将空闲内存组织为不同阶数的块，每个块包含连续的 `2^order` 个页（如 `order=0` 为 1 页，`order=1` 为 2 页）
- **score_a（标准碎片化指数）** 取值范围 `-1000`（最优）到 `1000`（最差）：
 - **负值：**存在足够大的连续块，分配容易。
 - **正值：**需拆分更高阶块，反映外部碎片化程度。
- **score_b（不可用页指数）** 因碎片化导致**无法满足当前** `order` **请求的空闲页比例**（0-1000），值越高表示碎片化越严重
- **分配上下文（`alloc_context`）**

```

1 struct alloc_context {
2     struct zonelist *zonelist;    // 可分配zone列表
3     nodemask_t *nodemask;         // 允许的NUMA节点掩码
4     struct zoneref *preferred_zoneref; // 首选zone引用
5     int migratetype;              // 页面迁移类型
6     enum zone_type highest_zoneidx; // 最高可用zone类型索引
7     bool spread_dirty_pages;      // 是否分散脏页
8 };

```

- **来源：**来自内核函数 `get_page_from_freelist` 的参数，决定内存分配策略。
- **连续页信息（`contig_page_info`）**

```

1 struct contig_page_info {
2     unsigned long free_pages;      // 总空闲页数
3     unsigned long free_blocks_total; // 总空闲块数
4     unsigned long free_blocks_suitable; // 适合当前order的块数
5 };

```


- **用途**：临时存储zone内各order的空闲块统计，用于计算碎片化指数。

- **BPF映射定义**

```
1 BPF_HASH(pgdat_map, u64, struct pgdat_info); // 存储NUMA节点信息
2 BPF_HASH(zone_map, u64, struct zone_info);    // 存储zone信息
3 BPF_HASH(last_time_map, u64, u64);           // 记录上次采样时间
4 BPF_ARRAY(delay_map, int, 1);                 // 控制采样间隔（秒）
```

- **功能**：

- `pgdat_map` 和 `zone_map` ：以指针地址为键，存储节点和zone的详细信息。
- `last_time_map` 和 `delay_map` ：实现时间窗口采样，避免高频事件导致性能开销

`get_page_from_freelist` 是 Linux 内核伙伴系统（Buddy System）的核心函数，负责在快速路径（fast path）中尝试从空闲内存链表中分配物理页面。以下是其作用与eBPF kprobe监控机制的详细解析

1. `get_page_from_freelist` 的核心功能

- 物理页分配：从伙伴系统的空闲链表中快速分配连续的物理内存页，满足内核或进程的内存请求。
- 性能优化：作为内存分配的“快速路径”，避免直接进入复杂的慢速路径（如内存回收、压缩等）。
- NUMA与区域感知：结合NUMA架构，优先从本地节点的内存区域（zone）分配，减少跨节点访问延迟。

2. eBPF kprobe的触发与回调机制

- 动态插桩：通过kprobe机制，eBPF程序将`kprobe__get_page_from_freelist`函数挂钩到`get_page_from_freelist`的入口，每次调用`get_page_from_freelist`时，会先执行该回调函数。
- 回调条件：
 - 时间过滤：通过`last_time_map`和`delay_map`控制采样频率，避免高频触发影响性能。
 - 参数传递：回调函数通过`struct pt_regs *ctx`和函数签名（如`alloc_context *ac`）获取内核函数的原始参数。

3. kprobe如何检测内存zone和order

(1) 内存区域（zone）检测

- **遍历备用区域列表**：

通过`alloc_context->preferred_zoneref`获取首选NUMA节点，遍历其所有内存区域（如DMA、Normal），检查以下条件：

- 水线检查：`zone_watermark_fast`判断当前区域的空闲页是否满足请求的阶数（order）和水线标记（如`ALLOC_WMARK_LOW`）。
- cpuset权限：若启用cpuset，检查当前进程是否被允许从该节点分配内存（`__cpuset_zone_allowed`）。

- 脏页限制：若请求用于文件写缓存（__GFP_WRITE），检查节点的脏页比例是否超限（node_dirty_ok）。

• 数据采集：

使用bpf_probe_read_kernel安全读取zone结构中的以下字段：

- zone->free_area[order].nr_free：各阶空闲页块数量。
- zone->name：区域名称（如"DMA"、"Normal"）。
- zone->spanned_pages/present_pages：区域总页数和实际可用页数。

(2) 阶数 (order) 检测

- 连续页块分析：fill_contig_page_info函数遍历所有阶数（0~MAX_ORDER），统计：
 - free_blocks_total：总空闲块数。
 - free_blocks_suitable：满足当前阶数的连续块数。
 - free_pages：总空闲页数。
- 碎片化评分：
 - unusable_free_index：计算因碎片化无法分配的内存比例（score_b）。总空闲页数减去能满足需求的连续页数/ 总空闲页数
 - __fragmentation_index：评估外碎片化程度（score_a），负值表示内存充足。

4. 检测结果存储与用途

- BPF映射存储：
 - zone_map：按区域和阶数存储碎片化评分及空闲页信息。
 - pgdat_map：记录NUMA节点元数据（如节点ID、区域数量）。
- 用户态分析：用户态程序读取这些映射，生成内存健康状态报告或触发告警。

5. 与传统监控的差异

特性	eBPF kprobe监控	传统工具（如/proc/buddyinfo）
实时性	纳秒级精度（ bpf_ktime_get_ns ）	秒级轮询
开销	低（JIT编译+采样控制）	高（频繁读取文件系统）
数据维度	多维度（碎片化评分、NUMA本地性等）	仅空闲页块数量
安全性	验证器确保安全，避免内核崩溃	依赖内核接口稳定性

get_page_from_freelist通过**高效遍历NUMA区域、水线检查、碎片优化**实现快速分配，而eBPF kprobe通过**动态插桩、安全内存访问、实时评分**对其行为进行监控。两者结合为内存调优与故障诊断提供了底层支持，尤其适用于需要低开销、高精度的生产环境。

unusable_free_index(order,struct contig_page_info*info):

计算当前内存区域中**无法满足指定阶（order）连续内存请求的空闲页面比例**，返回值范围 0-1000（相当于百分比×10）。

参数

- order
- : 请求的内存块阶数（如 order=2 表示请求 4 页连续内存）。
- info
- : 包含空闲页面统计信息的 contig_page_info结构体。

逻辑解析

```
1 if (info->free_pages == 0)
2     return 1000; // 无空闲页，完全不可用
3 return div_u64(
4     (info->free_pages - (info->free_blocks_suitable << order)) * 1000ULL,
5     info->free_pages);
```

分子: 总空闲页数减去能满足需求的连续页数（free_blocks_suitable << order）。

分母: 总空闲页数。

结果: 若 free_blocks_suitable=0（无满足需求的块），返回 1000（100%不可用）；若全部空闲页均可用，返回 0。

_fragmentation_index（unsigned int order,struct contig_page_info*info）：

这个代码片段是 Linux 内核中计算**内存碎片化指数**的核心算法，用于评估伙伴系统（buddy allocator）中特定内存区域（zone）的碎片化程度。以下是对其逐层解析：

1. 公式分解

代码的逻辑可以拆解为以下数学表达式：

¹ 碎片化指数 = 1000 - [(1000 + (总空闲页数 × 1000 / 请求页数)) / 总空闲块数]

其中：

- info->free_pages：当前 zone 的所有空闲页面总数（含碎片）。
- requested：请求的连续页数（1UL << order，如 order=2 表示请求 4 页）。
- info->free_blocks_total：所有阶（order）的空闲块数总和。

2. 计算步骤

1.
- div_u64(info->free_pages * 1000ULL, requested)
- 计算“总空闲页数可满足多少个请求”的千倍值（×1000 用于避免浮点运算）。

◦ 例如：若 free_pages=1000, requested=4（order=2），结果为 250,000（即 250 个请求 ×1000）。
2.
- 1000 + 步骤1结果
- 添加基线值 1000，防止后续除法结果过小。
3.
- div_u64(步骤2结果, info->free_blocks_total)
- 将调整后的值除以 总空闲块数，得到 每块的平均可分配能力。

◦ 若块数少但总页数多（即存在大块连续内存），此值较高；反之碎片化时较低。
4.
- 1000 - 步骤3结果
- 最终指数范围 0~1000：

▪ 接近 1000：高碎片化（分配失败主因是碎片）。

▪ 接近 0：低碎片化（分配失败主因是内存不足）。

3. 设计意图

- 量化碎片影响：通过对比“理论可分配量”与“实际块数分布”，反映碎片化对连续内存分配的阻碍程度。

• 触发内存整理：内核默认当指数 >500 时触发 kswapd 或内存压缩（compaction）。

• 动态调整阈值：可通过 /proc/sys/vm/extfrag_threshold 修改敏感度（默认 500）。

4. 示例场景

假设某 zone 的 contig_page_info 数据：

- free_pages=1000（总空闲页）

• free_blocks_total=10（总块数）

• requested=4（order=2）

计算过程：

1. $1000 \times 1000 / 4 = 250,000$
2. $1000 + 250,000 = 251,000$
3. $251,000 / 10 = 25,100$
4. $1000 - 25,100 \rightarrow$ 溢出处理为 0（实际内核会限制范围）

若 free_blocks_suitable > 0，直接返回 -1000（无需关心碎片）。

5. 关联函数

- fill_contig_page_info()：填充 free_pages 和 free_blocks_total 数据。

• __fragmentation_index()：封装此计算逻辑，处理边界条件（如无空闲块时返回 0）。

此算法是 Linux 内存管理的关键指标，直接影响性能优化策略（如大页分配、NUMA 调度等）。

fill_contig_page_info(struct zone*zone,unsigned int suitable_order,struct contig_page_info *info):

遍历伙伴系统的所有阶，使用**bpf_probe_read_kernel()**:安全读取内核内存，获取**当前阶数order**的空闲块数量nr_free。

以下是 fill_contig_page_info 函数的逐行解析，该函数用于统计 Linux 内核伙伴系统中特定内存区域（zone）的连续空闲页块信息：

函数定义与初始化

```
1 static void fill_contig_page_info(struct zone *zone,
2                                   unsigned int suitable_order,
3                                   struct contig_page_info *info) {
```

- **参数：**

- zone：指向内存区域（struct zone）的指针，表示待分析的 NUMA 节点或 UMA 内存区域。
- suitable_order：请求的连续页块阶数（如 order=2 表示需要 4 页连续内存）。
- info：输出参数，存储统计结果的 contig_page_info 结构体。

```
1     unsigned int order;
2     info->free_pages = 0;
3     info->free_blocks_total = 0;
4     info->free_blocks_suitable = 0;
```

- **初始化：**

- order：循环变量，遍历伙伴系统的所有阶（0 到 MAX_ORDER）。
- 清零输出结构的字段：总空闲页数、总空闲块数、满足需求的连续块数。

遍历伙伴系统的所有阶

```
1     for (order = 0; order <= MAX_ORDER; order++) {
2         unsigned long blocks;
3         unsigned long nr_free;
```

- **循环范围：**从最小阶（0）到最大阶（MAX_ORDER，通常为 10），覆盖所有可能的空闲块大小。

```
1     bpf_probe_read_kernel(&nr_free, sizeof(nr_free),
2                             &zone->free_area[order].nr_free);
3     blocks = nr_free;
```

- **读取空闲块数：**

- bpf_probe_read_kernel：安全读取内核内存，获取当前阶（order）的空闲块数量 nr_free。
- blocks：存储当前阶的空闲块数（如阶 2 有 3 个空闲块）。

统计全局空闲资源

```
1     info->free_blocks_total += blocks;
2     info->free_pages += blocks << order;
```

- **累加统计值：**

- free_blocks_total：所有阶的空闲块数总和（如阶 0 有 10 块 + 阶 1 有 5 块 → 15 块）。
- free_pages：总空闲页数，按阶转换为页数累加（如阶 2 的 3 块贡献 $3 \ll 2 = 12$ 页）。

统计满足需求的连续块

```
1     if (order >= suitable_order)
2         info->free_blocks_suitable += blocks << (order - suitable_order);
```

- **条件判断：**仅统计阶数 $\geq$ suitable_order 的块。
- **转换计算：**将高阶块数转换为等效的 suitable_order 阶块数。
 - 例如，suitable_order=2 时，阶 3 的 1 块（8 页）等效于 $1 \ll (3-2) = 2$ 个 4 页块。

示例场景

假设 suitable_order=2，某 zone 的空闲块分布如下：

- 阶 0：10 块（每块 1 页）→ 贡献 free_pages=10, free_blocks_total=10
- 阶 2：3 块（每块 4 页）→ 贡献 free_pages=12, free_blocks_total=3, free_blocks_suitable=3
- 阶 3：1 块（每块 8 页）→ 贡献 free_pages=8, free_blocks_total=1, free_blocks_suitable=2

最终结果：

- free_pages=30
 - free_blocks_total=14
 - free_blocks_suitable=5
-

功能关联

- **与碎片化指数计算**：此函数为 `__fragmentation_index` 和 `unusable_free_index` 提供基础数据，用于量化内存碎片化程度。
- **性能优化**：通过 BPF 安全读取内核数据，避免直接访问风险，适用于动态监控工具（如 `bpfftrace`）。

kprobe_get_page_from_freelist():

监控 `get_page_from_freelist` 函数的执行情况，并收集内存分配的相关统计信息，`ctx`：保存寄存器上下文的 `pt_regs` 结构，用于访问函数调用时的寄存器状态；`gfp_mask`：内存分配标志；`order`：请求内存块结阶数；`alloc_flags`：分配控制标志；`ac`：分配上下文，包括 NUMA 节点、zone 优先级信息。

1. 函数定义与时间控制逻辑

```
1 int kprobe__get_page_from_freelist(struct pt_regs *ctx, gfp_t gfp_mask,
2                                   unsigned int order, int alloc_flags,
3                                   const struct alloc_context *ac) {
```

• 参数说明：

- `ctx`：保存寄存器上下文的 `pt_regs` 结构，用于访问函数调用时的寄存器状态。
- `gfp_mask`：内存分配标志（如 `GFP_KERNEL`）。
- `order`：请求的内存块阶数（ 2^{order} 页）。
- `alloc_flags`：分配控制标志（如 `ALLOC_HARDER`）。
- `ac`：分配上下文，包含 NUMA 节点、zone 优先级等信息。

```
1  u64 *last_time, current_time;
2  current_time = bpf_ktime_get_ns(); // 获取当前纳秒级时间戳
3  last_time = last_time_map.lookup(&current_time);
4  int key = 0;
5  int *delay_ptr = delay_map.lookup(&key);
6  int delay;
7  if (delay_ptr) {
8      delay = *delay_ptr; // 从 delay_map 读取延迟阈值（秒）
9  }
10 if (last_time && (current_time - *last_time < delay * 1000000000)) {
11     return 0; // 若未超过延迟间隔，直接退出
12 }
```

• 时间控制机制：

- 通过 last_time_map 记录上一次执行时间戳，delay_map 存储最小采样间隔（秒）。
- 避免高频触发导致性能开销，仅在时间间隔超过 delay 时继续执行。

2. 遍历内存区域 (zone) 并收集信息

```
1 struct pglist_data *pgdat;
2 struct zone *z;
3 struct zoneref *zref;
4 int i, tmp, index, res;
5 unsigned int a_order;
6
7 pgdat = ac->preferred_zoneref->zone->zone_pgdat; // 获取首选 NUMA 节点的 pglist_data
```

- **pgdat**: 指向当前 NUMA 节点的内存描述符，包含所有 zone 的链表。

```
1 for (i = 0; i < MAX_NR_ZONES; i++) {
2     struct zone_info zone_data = {};
3     struct pgdat_info pgdat_data = {};
4     struct pgdat_info *a_pgdat;
5     struct pglist_data *pgdata;
6     u64 node_key, zone_key;
7     zref = &pgdat->node_zonelists[ZONELIST_FALLBACK]._zonerefs[i];
8     z = zref->zone;
9     if (!z)
10         continue; // 跳过无效 zone
```

- **遍历 zone**:
 - node_zonelists[ZONELIST_FALLBACK] 是后备 zone 列表，按分配优先级排序。
 - 检查 zone 有效性，无效时跳过。

3. 更新 NUMA 节点 (pgdat) 信息

```
1 pgdata = z->zone_pgdat;
2 if (!pgdata)
3     continue;
4 node_key = (u64)pgdata;
5 a_pgdat = pgdat_map.lookup(&node_key);
6 if (!a_pgdat) {
```

```

7     pgdat_data.pgdat_ptr = (u64)pgdata->node_start_pfn; // 节点起始页帧号
8     pgdat_data.nr_zones = pgdata->nr_zones;             // zone 数量
9     pgdat_data.node_id = pgdata->node_id;               // NUMA 节点 ID
10    pgdat_map.update(&node_key, &pgdat_data);          // 写入 pgdat_map
11 }

```

- **pgdat_map:**

- 存储 NUMA 节点的关键信息，如页帧范围、zone 数量等。
- 首次访问时初始化并更新。

4. 更新 zone 信息

```

1     zone_data.zone_ptr = (u64)z;
2     zone_data.zone_start_pfn = z->zone_start_pfn;      // zone 起始页帧号
3     zone_data.spanned_pages = z->spanned_pages;         // 物理跨度页数
4     zone_data.present_pages = z->present_pages;        // 实际可用页数
5     zone_data.node_id = z->zone_pgdat->node_id;         // 所属节点 ID
6     bpf_probe_read_kernel_str(&zone_data.name, sizeof(zone_data.name), z->name); // 安全
    读取 zone 名称

```

- **zone_data:**

- 记录 zone 的物理内存布局和属性，通过 bpf_probe_read_kernel_str 安全读取内核数据。

5. 统计各阶 (order) 内存碎片化指数

```

1     for (a_order = 0; a_order <= MAX_ORDER; ++a_order) {
2         zone_data.order = a_order;
3         zone_key = zone_data.zone_ptr + zone_data.order; // 生成唯一 zone+order 键
4
5         struct contig_page_info ctg_info;
6         fill_contig_page_info(z, a_order, &ctg_info);    // 填充连续页块信息
7         zone_data.free_blocks_suitable = ctg_info.free_blocks_suitable; // 满足需求的块数
8         zone_data.free_blocks_total = ctg_info.free_blocks_total;        // 总空闲块数
9         zone_data.free_pages = ctg_info.free_pages;          // 总空闲页数
10
11        tmp = unusable_free_index(a_order, &ctg_info);    // 计算不可用内存比例 (0-1000)
12        zone_data.score_b = tmp;
13        index = __fragmentation_index(a_order, &ctg_info); // 计算碎片化指数 (-1000~1000)
14        zone_data.score_a = index;

```



```

15
16     zone_map.update(&zone_key, &zone_data); // 更新 zone_map
17     zone_key++;
18 }

```

- **关键操作：**

- fill_contig_page_info：统计当前 zone 中不同阶的空闲块信息。
- unusable_free_index：评估无法满足 a_order 需求的空闲页比例。
- __fragmentation_index：量化内存碎片化程度（负值表示低碎片）。
- 结果写入 zone_map，供用户空间工具分析。

6. 更新时间戳并返回

```

1  last_time_map.update(&current_time, &current_time); // 记录本次执行时间
2  return 0; // 返回成功
3  }

```

- **收尾工作：**

- 更新 last_time_map 以控制下次触发间隔。
- 返回 0 表示正常执行完毕。

1. **动态采样控制：**通过时间戳避免高频触发，降低性能开销。
2. **NUMA 与 Zone 遍历：**按优先级扫描内存区域，收集物理布局信息。
3. **碎片化评估：**计算各阶内存的连续性和可用性指标。
4. **安全数据访问：**使用 bpf_probe_read_kernel_str 避免直接解引用内核指针。
5. **eBPF Map 交互：**通过 pgdat_map、zone_map 等与用户空间共享数据。

4.extfraginfo.c:

函数核心：

1. 核心功能

- **监控内存碎片化分配：**通过TRACEPOINT_PROBE(kmem, mm_page_alloc_extfrag)挂钩内核的 mm_page_alloc_extfrag事件，该事件在内存分配因碎片化需要降级（fallback）时触发。
- **数据记录：**捕获每次碎片化分配的详细信息，包括：
 - **物理页帧号（PFN）：**args->pfn，标识分配的物理内存位置。
 - **分配阶数：**args->alloc_order（请求的连续页块阶数）和args->fallback_order（实际分配的阶数，可能更小）。（可能因为不足而降级）

- 。进程信息：进程PID和名称（pcomm），用于关联分配行为到具体进程。

2. 关键组件

(1) BPF数据结构

- **counts_map**（哈希表）：**BPF_HASH**(counts_map, pid_t, struct data_t)
以进程PID为键，存储struct data_t类型的值，记录每个进程的碎片化分配统计（如分配次数、阶数变化等）。
- last_time_map和delay_map：
控制采样频率，避免高频事件导致性能开销。delay_map存储时间间隔（秒），last_time_map记录上次触发时间。

(2) 时间过滤逻辑

```
1 if (last_time && (current_time - *last_time < delay * 1e9))  
2     return 0; // 未达到采样间隔则跳过
```

- 通过bpf_ktime_get_ns()获取纳秒级时间戳，确保仅间隔足够时间（delay秒）时处理事件。

(3) 进程上下文获取

- bpf_get_current_pid_tgid() >> 32：提取当前进程的PID（高32位）。（BPF辅助函数）
- bpf_get_current_comm()：读取进程名称（如bash或nginx），存储到pcomm字段。

3. 数据更新逻辑

- 新进程记录：若counts_map中无对应PID，初始化struct data_t，记录首次分配的PFN、阶数和进程名。（使用args参数获得：在 `TRACEPOINT_PROBE(kmem, mm_page_alloc_extfrag)` 中，args 参数是由内核Tracepoint框架自动填充的结构体指针，其内容来源于内核预定义的 `mm_page_alloc_extfrag` 跟踪点）
- 已有进程更新：若PID已存在，递增count字段并更新分配阶数和PFN，反映最新的碎片化分配行为。

4. 技术背景

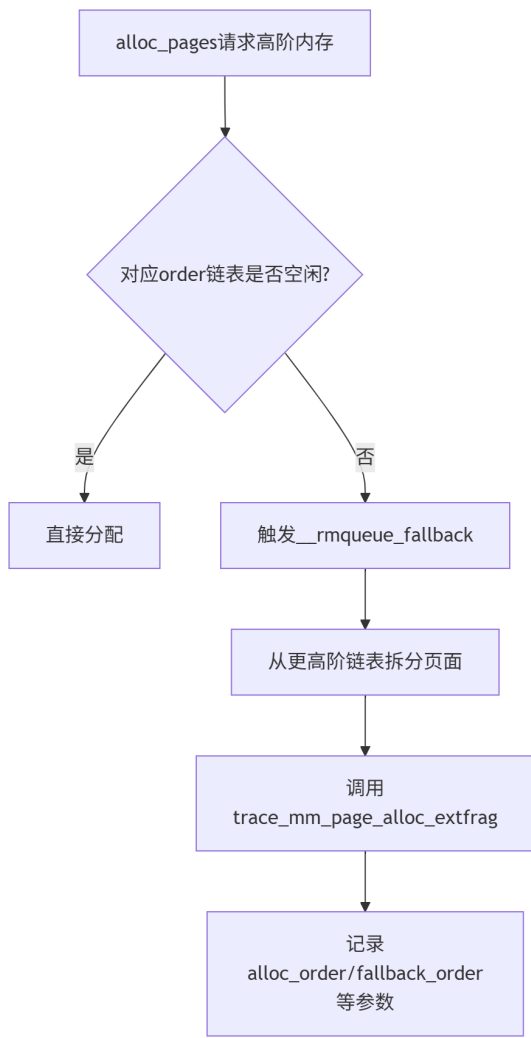
- **mm_page_alloc_extfrag**事件：

当伙伴系统无法直接满足请求的连续页块（如alloc_order=3请求8页），需从更高阶空闲链表中拆分时触发，表明存在内存碎片化问题。

（高阶内存分配失败时的降级分配当内核通过伙伴系统（Buddy System）尝试分配连续的高阶内存块（如 2^{order} 个页面）时，如果对应阶数的空闲链表为空，则会尝试从更高阶的链表中拆分页面。例如：

- 请求分配 8KB（order=1）的内存，但 order=1 的链表为空
- 内核从 order=2（16KB）的链表中拆分出 8KB 进行分配，剩余 8KB 插入 order=1 的链表
- 此过程会触发 `mm_page_alloc_extfrag`，记录降级分配事件

外部碎片（External Fragmentation）产生时降级分配会导致更高阶的连续内存块被拆分，增加外部碎片（即剩余内存块物理不连续，无法满足后续的高阶分配需求））



- eBPF的TRACEPOINT_PROBE：
比kprobe更稳定的内核事件挂钩方式，基于预定义的tracepoint（如kmem子系统），无需担心内核函数签名变化。

5. 应用场景

- 性能调优：统计高频碎片化分配的进程，优化内存分配策略（如调整/proc/sys/vm/zone_reclaim_mode）。
- 故障诊断：结合PFN和阶数变化，分析内存碎片化热点。
- 安全监控：检测异常进程的大规模内存分配行为（如拒绝服务攻击）。

6. 示例输出分析

假设某次事件捕获：

```
1 data_t = {
2     pfn: 0x123456,
3     alloc_order: 3, // 请求8页
4     fallback_order: 1, // 实际分配2页（降级）
5     pid: 512,
```

```
6     pcomm: "nginx"
7 }
```

表明Nginx进程因碎片化无法分配8页，降级为2页，需关注NUMA或水线配置。

总结

这段代码通过eBPF的tracepoint机制，实现了对内存碎片化分配的低开销监控，为性能优化和问题定位提供数据支撑。其设计结合了**时间采样**、**进程上下文捕获**、**动态数据更新**三大机制，是内存管理的实用调试工具。

用途和意义：

这段BPF程序通过监控 `mm_page_alloc_extfrag` 跟踪点，专门用于分析和记录Linux内核中因**内存碎片化导致的高阶内存降级分配事件**。其核心意义和实现逻辑可分为以下几个层面：

1. 监控的内存分配场景

程序触发的条件是：当内核尝试分配**连续内存块**（如 `2^alloc_order` 页）时，由于对应阶数的空闲链表不足，被迫从更高阶的链表中拆分页面（例如：请求8KB内存但`order=1`链表为空，只能从`order=2`的16KB链表中拆分）。这种操作称为**fallback分配**，会加剧内存外部碎片化。

关键字段的含义：

- `alloc_order`：进程最初请求的内存块阶数（如 `order=1` 表示8KB）。
- `fallback_order`：实际分配的更高阶数（如 `order=2` 表示从16KB拆分）。
- `pfn`：分配的物理页帧号，标识内存位置。

2. 数据收集的意义

通过 `data_t` 结构体记录的信息，可以实现：

- 碎片化量化分析** `fallback_order > alloc_order` 的差值直接反映碎片化程度。例如：
 - 频繁的 `order=3 -> order=4` 降级（32KB请求需拆分64KB块）表明高阶内存严重碎片化。
- 进程级责任定位** 结合 `pid` 和 `pcomm`（进程名），可识别哪些进程频繁触发降级分配，例如：
 - 数据库服务可能因大页（THP）分配失败导致碎片化。
 - 网络栈中 `order>=3` 的页面分配可能因PCP缓存不足引发zone锁竞争。
- 性能瓶颈诊断** 降级分配会导致：
 - 额外的页面拆分开销（CPU周期增加）
 - 高阶连续内存的永久性减少（可能触发后续内存规整）

3. 程序实现的技术细节

- 采样控制** 通过 `delay_map` 和 `last_time_map` 实现时间窗口采样，避免高频事件导致性能开销。
- 进程上下文关联** 使用 `bpf_get_current_pid_tgid()` 和 `bpf_get_current_comm()` 动态绑定事件到进程，补充了Tracepoint原生参数未包含的进程信息。
- 数据聚合** `counts_map` 按PID聚合事件次数，统计每个进程的碎片化分配频率，而非仅记录单次事件。

4. 实际应用场景

1. 性能调优

- 若某进程的 `count` 值异常高，可优化其内存分配模式（如改用slab或调整THP配置）。
- 结合 `/proc/buddyinfo` 观察剩余高阶内存块，判断是否需要调整水位线或NUMA策略。

2. 内存泄露辅助判断

长期累积的 `count` 增长且无释放记录，可能预示内存泄露（需结合 `mm_page_free` 事件分析）。

3. 内核参数优化

- 调整 `/proc/sys/vm/extfrag_threshold` 控制碎片化处理策略。
- 修改PCP缓存大小（如 `/proc/sys/vm/percpu_pagelist_fraction` ）减少zone锁竞争。

5. 与内核机制的关联

- **伙伴系统反碎片策略**该程序监控的行为正是Linux迁移类型分组（migratetype）机制试图缓解的问题。当 `fallback_migratetype != alloc_migratetype` 时，说明内核从其他迁移类型"盗取"了页面，可能破坏反碎片设计。
- **THP（透明大页）影响**大页分配失败时会退化为小页分配，可能表现为 `alloc_order=9（2MB） -> fallback_order=0（4KB）`，此时需要关闭THP或优化工作负载。

总结

该BPF程序的核心价值在于：**将内存碎片化这一抽象问题转化为可量化的进程级指标**，通过 `data_t` 中的阶数差异、进程信息和统计计数，为系统管理员提供以下能力：

1. 定位碎片化热点进程
2. 评估内存子系统的健康度
3. 验证调优策略的实际效果（如调整PCP缓存后的 `count` 下降）

如需更深入的分析，可结合 `/proc/buddyinfo` 、 `sar -B` 和 `ftrace` 事件综合判断

5.Kprobe和tracepoint区别：

在 eBPF 中， `tracepoint_probe` 和 `kprobe` 是两种不同的动态追踪机制，它们在设计理念、使用场景、性能开销和稳定性等方面存在显著差异。以下结合你提供的代码示例，详细分析两者的区别：

1. 设计理念与接口类型

• Tracepoint

- **静态探针**：由内核开发者通过 `TRACE_EVENT()` 宏预定义（如 `kmem:mm_page_alloc_extfrag` ），提供稳定的 ABI（应用程序二进制接口）。
- **示例代码**：`TRACEPOINT_PROBE(kmem, mm_page_alloc_extfrag)` 直接挂钩到内核预定义的 `mm_page_alloc_extfrag` 事件，通过 `args->pfn` 等参数访问事件数据。
- **适用场景**：监控稳定的内核事件（如内存分配、系统调用），适合生产环境长期运行。

• Kprobe

- **动态探针**：可挂钩到任意内核函数（如 `get_page_from_freelist` ），无需内核预定义，依赖内核符号表。

- **示例代码:** `kprobe__get_page_from_freelist` 挂钩到伙伴系统的核心函数, 通过 `struct pt_regs` `*ctx` 和 `bpf_probe_read_kernel()` 读取函数参数和内存数据。
- **适用场景:** 调试私有或未公开的内核函数, 灵活性高但稳定性差。

2. 稳定性与兼容性

- **Tracepoint**
 - **稳定:** 接口和参数在不同内核版本间保持一致, 适合编写可移植的 eBPF 程序。
 - **参数访问:** 直接通过 `args->field` 访问结构化数据 (如 `args->pfn`), 无需手动解析。
- **Kprobe**
 - **不稳定:** 函数名、参数或结构体可能随内核更新变化, 需依赖 `BPF_CORE_READ` 等宏处理兼容性问题。
 - **参数访问:** 需通过 `pt_regs` 或手动读取内存 (如 `bpf_probe_read_kernel(&nr_free, ...)`), 易因内核变更失效。

3. 性能开销

- **Tracepoint**
 - **低开销:** 编译时优化为 `NOP` 指令, 未激活时几乎无性能影响; 激活后仅触发简单回调。
 - **数据流优化:** 原始参数直接传递给 eBPF 程序, 减少数据拷贝 (如 `raw_tracepoint` 性能更高)。
- **Kprobe**
 - **较高开销:** 动态插入断点指令 (如 `int3`), 触发中断和上下文切换; 优化版本 (如 `kprobe-optimized`) 通过跳转指令减少开销, 但仍劣于 Tracepoint。
 - **额外操作:** 需手动读取内存和寄存器, 增加 CPU 负载。

4. 使用场景对比

特性	Tracepoint	Kprobe
接口类型	静态 (内核预定义)	动态 (任意函数)
稳定性	高 (ABI 稳定)	低 (依赖符号表)
性能	低开销 (编译时优化)	较高开销 (断点中断)
参数访问	直接访问 <code>args->field</code>	需通过 <code>pt_regs</code> 或内存读取
生产环境适用性	推荐长期监控	仅限临时调试
示例代码事件	<code>mm_page_alloc_extfrag</code> (内存分配事件)	<code>get_page_from_freelist</code> (伙伴系统函数)

5. 代码示例分析

局限性:

- **覆盖范围有限:** Tracepoint 通常暴露高层事件 (如 `mm_page_alloc`) , 但不直接提供底层 `struct zone` 或 `struct pglist_data` 的完整信息。
- **数据粒度不足:** 例如, `mm_page_alloc` Tracepoint 可能仅提供页帧号 (`pfn`) 和分配阶 (`order`) , 而无法直接获取 `zone->free_pages` 或 NUMA 节点拓扑

1. Tracepoint 示例

- 挂钩到 `kmem:mm_page_alloc_extfrag` , 直接使用 `args->pfn` 等参数, 无需处理底层内存。
- 适合统计内存分配次数和进程信息, 稳定性高。

2. Kprobe 示例

- 挂钩到 `get_page_from_freelist` , 需通过 `bpf_probe_read_kernel()` 读取 `struct zone` 等复杂结构体。
- 适合深入分析伙伴系统的碎片化情况, 但需处理兼容性风险。

6._fragmentation_index (fraginfo) :

Linux内核中 `__fragmentation_index` 函数的缓冲公式设计并非凭空而来, 而是基于内存碎片化问题的数学建模和实际系统行为观察的综合结果。其设计参考了以下核心逻辑和背景:

1) . 数学建模: 量化碎片与内存利用率的关系

公式的核心是通过归一化空闲页与请求大小的比例, 结合空闲块总数, 动态反映碎片化对连续分配的影响:

```
1 1000 - div_u64((1000 + (div_u64(info->free_pages * 1000ULL, requested))), info->free_blocks_total)
```

- `info->free_pages * 1000ULL / requested` :
将总空闲页按请求大小归一化, 计算“理论可分配次数”。例如, 若总空闲页为20页, `order=2` (需4页) , 则理论可分配次数为5次 (`20/4=5`) 。乘以1000是为了避免浮点运算, 使用整数计算提高性能。
- **加1000缓冲:**
防止 `free_blocks_total` 过小 (如1) 时计算结果过大, 导致数值不稳定。这一缓冲值通过实验确定, 平衡了极端情况的敏感性和普适性。
- **按空闲块总数平均:** 反映碎片化对整体利用率的影响。若空闲块多但分散 (碎片化高) , 则平均值会降低; 反之, 若块数少但连续 (碎片化低) , 平均值较高。
- **反向刻度 (1000 - 结果) :** 将结果映射到0~1000范围, 使返回值与碎片化程度正相关 (值越大越碎片化) , 符合直觉认知。

2) . 实际系统行为的观察与调优

- **极端情况处理:**

- 若 `free_blocks_suitable > 0`（存在满足需求的连续块），直接返回 `-1000`，表示无碎片化压力。这一设计源于实际场景中，连续块的存在是分配成功的关键，无需复杂计算。
- 若 `free_blocks_total = 0`（无空闲块），返回0，避免无意义计算。这符合伙伴系统“无内存即失败”的基本逻辑。
- **阈值与触发机制：**
默认阈值500（通过 `/proc/sys/vm/extfrag_threshold` 可调）是内核长期调优的结果。当返回值超过500时，触发内存规整（compaction），表明碎片化是分配失败的主因；低于500则认为是内存不足。

其设计目标是：以最小计算开销动态评估碎片化，为内核决策（如触发compaction或回收）提供量化依据。

7.get_page_from_freelist（fraginfo）：

`get_page_from_freelist` 是 Linux 内核伙伴系统（Buddy System）中的一个核心函数，主要用于从物理内存的空闲列表中快速分配连续的物理内存页。它在内存分配的快速路径（Fast Path）中被调用，是伙伴系统实际执行内存分配的关键环节。

作用

1. 遍历空闲内存区域

函数会扫描 NUMA 节点中的内存区域（zone），根据分配标志（`gfp_mask`）和分配阶（`order`），寻找满足条件的空闲内存块。若首选区域（`preferred_zone`）无法分配，则按备用区域列表（`zonelist`）顺序尝试其他区域。

2. 检查分配条件

- **水位线检查：**通过 `zone_watermark_fast` 判断当前区域的水位是否足够（如低水位、最低水位等），若不足可能触发回收或切换区域。
- **CPU 和 NUMA 限制：**检查 `cpuset` 是否允许当前进程从目标节点分配内存，以及 NUMA 节点的本地性（避免跨节点分配导致性能下降）。
- **脏页限制：**若分配的是文件缓存页（`__GFP_WRITE`），需确保节点脏页数量未超限。

3. 调用伙伴系统分配内存

若条件满足，最终调用 `rmqueue` 从伙伴系统的空闲链表中分配指定阶数的连续物理页，并初始化页属性（如通过 `prep_new_page`）。

调用时机

`get_page_from_freelist` 在以下场景被调用：

1. 快速路径分配

当内核通过 `__alloc_pages_nodemask` 分配内存时，首先尝试快速路径（低水位分配），此时直接调用 `get_page_from_freelist`。若成功则立即返回，否则进入慢速路径（如回收内存、压缩等）。

2.

避免碎片的分配请求

当分配标志包含 `ALLOC_NOFRAGMENT` 时，函数会优先从本地节点分配，避免内存碎片。若失败则放宽限制重试。

3. 特定迁移类型分配

根据 `migratetype`（如不可移动页 `MIGRATE_UNMOVABLE` 或可移动页 `MIGRATE_MOVABLE`），从对应的空闲链表中分配内存，以支持反碎片化机制。

关键流程示例

- 1. 输入参数：**包括分配阶 `order`、内存区域标识 `high_zoneidx`、迁移类型 `migratetype` 等。
- 2. 遍历备用区域：**从 `zonelist` 中依次检查每个区域的水位、CPU 限制、脏页状态等。
- 3. 分配尝试：**若条件满足，调用 `rmqueue` 分配内存；若失败则继续遍历或触发慢速路径。

`get_page_from_freelist` 是伙伴系统中**高效分配连续物理内存的核心函数**，其调用时机和逻辑紧密关联内核的内存管理策略（如水位控制、碎片避免、NUMA 优化等）。它在快速路径中扮演关键角色，确保在内存充足时快速响应分配请求，失败时则交由慢速路径处理复杂场景。

8.Linux系统内存分配机制：

伙伴系统(Buddy System)是Linux内核中用于管理物理内存的核心算法，它通过将内存划分为不同大小的块并以特定方式组织这些块，有效地解决了内存分配中的**外部碎片问题**。

一、伙伴系统的基本概念与数据结构

1. 内存区域(Zone)划分

Linux内核将物理内存划分为不同的管理区(Zone)，主要有三种类型：

- `ZONE_DMA`：包含低于16MB的内存页框，用于直接内存访问(DMA)操作
- `ZONE_NORMAL`：包含高于16MB且低于896MB的内存页框，是内核正常使用的区域
- `ZONE_HIGHMEM`：包含从896MB及之上的内存页框，用于管理高端内存

每个zone结构中都维护着一个 `free_area` 数组，用于管理不同大小的空闲内存块。

2. 阶数(Order)与块大小

伙伴系统将内存块组织为2的幂次方大小，称为"阶"(order)：

- 0阶：1个页面(通常4KB)
- 1阶：2个页面(8KB)
- ...
- `MAX_ORDER`阶(通常为10阶)：1024个页面(4MB)

每个阶对应一个空闲链表，管理该大小的空闲内存块。内核通过 `free_area[MAX_ORDER]` 数组来维护这些链表。

3. 伙伴关系定义

两个内存块互为"伙伴"需要满足以下条件：

- 1. 物理地址连续**
- 2. 由同一父块分裂产生**
- 3. 合并后可形成完整的更高阶块**

二、内存分配的核心流程

1. 分配入口函数

内存分配的主要入口函数是 `alloc_pages()`，它接收两个关键参数：

- `gfp_mask`：分配标志，指定分配行为和区域
- `order`：请求的阶数，表示需要 2^{order} 个连续物理页面

```
struct page *alloc_pages(gfp_t gfp_mask, unsigned int order);
```

2. 区域选择

内核根据 `gfp_mask` 中的区域修饰符选择优先扫描的Zone：

- `__GFP_DMA`：指定从DMA区分配
- `__GFP_HIGHMEM`：指定从高端内存区分配
- 默认情况下从Normal区分配

选择过程通过 `gfp_zone()` 函数实现，它从分配掩码中计算出zone的索引。

3. 理想情况下的分配路径

在内存充足且无碎片的理想情况下，分配流程如下：

1. 匹配请求阶数：首先检查目标区域的 `order` 阶空闲链表(`free_area[order].free_list`)是否有可用块
2. 直接分配：如果找到匹配块，从链表中移除并标记为已分配，返回第一个页面的 `struct page` 指针
3. 分裂高阶块：如果当前阶无可用块，向上查找更高阶(如`order+1`)的空闲链表
4. 递归分裂：
 - 将高阶块对半分裂为两个当前阶块
 - 一个加入当前阶空闲链表
 - 另一个分配给用户
5. 示例：请求8个页面(3阶)但3阶无空闲：
 - 查找4阶链表，若有则分裂为两个3阶块(各8页)
 - 分配一个3阶块，另一个加入3阶空闲链表

4. 分配流程图解

```
1 开始
2  |
3  v
4  检查order阶空闲链表是否有可用块
5  |--有--> 分配并返回
6  |
7  无
8  |
```

```

9    v
10   检查order+1阶链表
11   |--有--> 分裂为两个order阶块
12   |         |--> 一个分配给用户
13   |         |--> 另一个加入order阶链表
14   |
15   无
16   |
17   v
18   继续向上检查更高阶(order+2, order+3...)
19   |--有--> 递归分裂直到得到order阶块
20   |
21   无
22   |
23   v
24   分配失败

```

三、关键数据结构与函数实现

1. 主要数据结构

- **struct zone**: 内存区域描述符, 包含 `free_area[MAX_ORDER]` 数组
- **struct free_area**: 管理特定阶的空闲块:

```

1  struct free_area {
2      struct list_head free_list[MIGRATE_TYPES]; // 空闲链表
3      unsigned long nr_free; // 空闲块数量
4  };

```

- **struct page**: 页面描述符, 包含链表指针和其他元数据

2. 核心分配函数

`__alloc_pages()` 是伙伴系统的核心分配函数, 其主要逻辑包括:

1. 通过 `prepare_alloc_pages()` 准备分配上下文
2. 调用 `get_page_from_freelist()` 尝试快速分配
3. 处理各种特殊情况(如内存不足、需要回收等)

3. 快速路径分配

理想情况下的快速分配路径由 `get_page_from_freelist()` 实现:

```

1  // 简化逻辑

```

```
2 page = get_page_from_freelist(alloc_mask, order, alloc_flags, &ac);
3 if (likely(page))
4     goto out; // 分配成功
```

四、保证连续性的机制

伙伴系统通过以下设计保证分配的物理内存连续性：

1. 块大小对齐：每个阶的块起始地址必须是块大小的整数倍
 - 例如16页(4阶)块的起始地址必须是 $16 \times 4K = 64K$ 的倍数
2. 伙伴合并规则：只有满足特定条件的块才能合并
3. 迁移类型隔离：将页面按迁移类型(MIGRATE_UNMOVABLE/MOVABLE/RECLAIMABLE)分类管理，减少碎片

五、性能优化措施

为提高分配效率，伙伴系统实现了多种优化：

1. 每CPU页缓存(Per-CPU Pageset, PCP)：针对单页(0阶)请求，优先从CPU本地缓存分配，减少全局锁竞争
2. 水位线控制：每个区域设置低、中、高三级水位线，仅在空闲页数高于最低水位时允许快速分配
3. 迁移类型分组：减少内存碎片，提高大块连续内存的可用性

六、实际分配示例

假设在x86_64系统(页大小4K)上请求分配256个连续页框(1MB)：

1. 计算阶数： $256 = 2^8 \rightarrow \text{order} = 8$
2. 检查目标zone的free_area[8].free_list
3. 若8阶链表有空闲块：
 - 从链表取出第一个块
 - 清除private字段(表示连续页属于哪个链表)
 - 返回该块的第一个页描述符
4. 若8阶无空闲：
 - 检查9阶链表(512页)
 - 将512页块分裂为两个256页块
 - 一个分配给请求
 - 另一个加入8阶链表

七、与相关系统的协作

伙伴系统不是独立工作的，它与内核其他内存管理机制紧密配合：

1. SLAB分配器：伙伴系统解决外部碎片问题，SLAB优化内部碎片管理
2. NUMA架构：通过pg_data_t结构实现多节点内存域管理
3. 页表系统：x86_64采用四级页表结构将虚拟地址转换为物理地址

总结

Linux伙伴系统通过阶数化的内存块管理、递归分裂与合并机制，以及精细的区域划分，实现了高效连续的物理内存分配。在理想情况下，它能够以 $O(1)$ 时间复杂度完成分配，同时通过伙伴合并策略有效减少内存碎片。理解伙伴系统的工作原理对于深入掌握Linux内存管理机制至关重要，也是进行内核级内存优化和调试的基础。

9.BCC作用:

BCC 在 eBPF 开发中的作用

1. 简化 eBPF 程序的编写

- eBPF 程序通常需要通过 C 语言或汇编语言编写，并与内核交互。而 BCC 提供了一个更为简洁的编程接口，允许开发者以 **Python** 或 **C**等更易用的语言编写 eBPF 程序。
- 通过 BCC，开发者可以避免直接处理内核细节，如内存管理和内核与用户空间的数据传输等。

2. 自动处理编译与加载

- BCC 自动处理 eBPF 程序的编译和加载过程。它能够将 eBPF 程序加载到内核，并确保其正确运行，无需开发者手动干预编译和加载步骤。
- BCC 提供了简单的 API，开发者只需关注编写程序的逻辑，而无需深入了解 eBPF 的低级细节。

3. 提供易用的工具集

- BCC 包含了许多已经编写好的 **eBPF 程序** 和 **分析工具**，这些工具可以直接用于 **系统监控、网络分析、性能调优** 等任务。
- 例如，`execsnoop` 可以用来监控系统中的进程启动，`opensnoop` 可以监控文件打开的系统调用，而 `trace` 工具可以用来跟踪内核函数调用等。

4. 性能监控与系统跟踪

- BCC 提供了一系列内建的 **性能监控** 和 **系统跟踪** 工具。通过 eBPF 程序，你可以动态地追踪内核和用户空间的行为，帮助开发者捕捉系统瓶颈、识别性能问题并优化系统。
- 比如，开发者可以使用 BCC 监控 TCP 网络流量，追踪系统调用，或者分析应用程序的内存使用等。

5. 内核与用户空间的数据传输

- eBPF 程序通常需要在 **内核空间** 和 **用户空间** 之间交换数据。BCC 提供了 **eBPF 映射 (maps)** 的接口，用来高效地存储和传递这些数据。
- 这些映射可以是散列表、队列或其他数据结构，BCC 使得开发者能够更轻松地实现数据交换和共享。

6. 便于调试和分析

- BCC 提供了 **实时输出** 和 **调试支持**，让开发者可以在开发过程中方便地查看 eBPF 程序的运行状态。
- 例如，开发者可以通过 BCC 动态监控 eBPF 程序的执行情况，查看是否存在错误或性能瓶颈。

BCC 的核心组件和工具

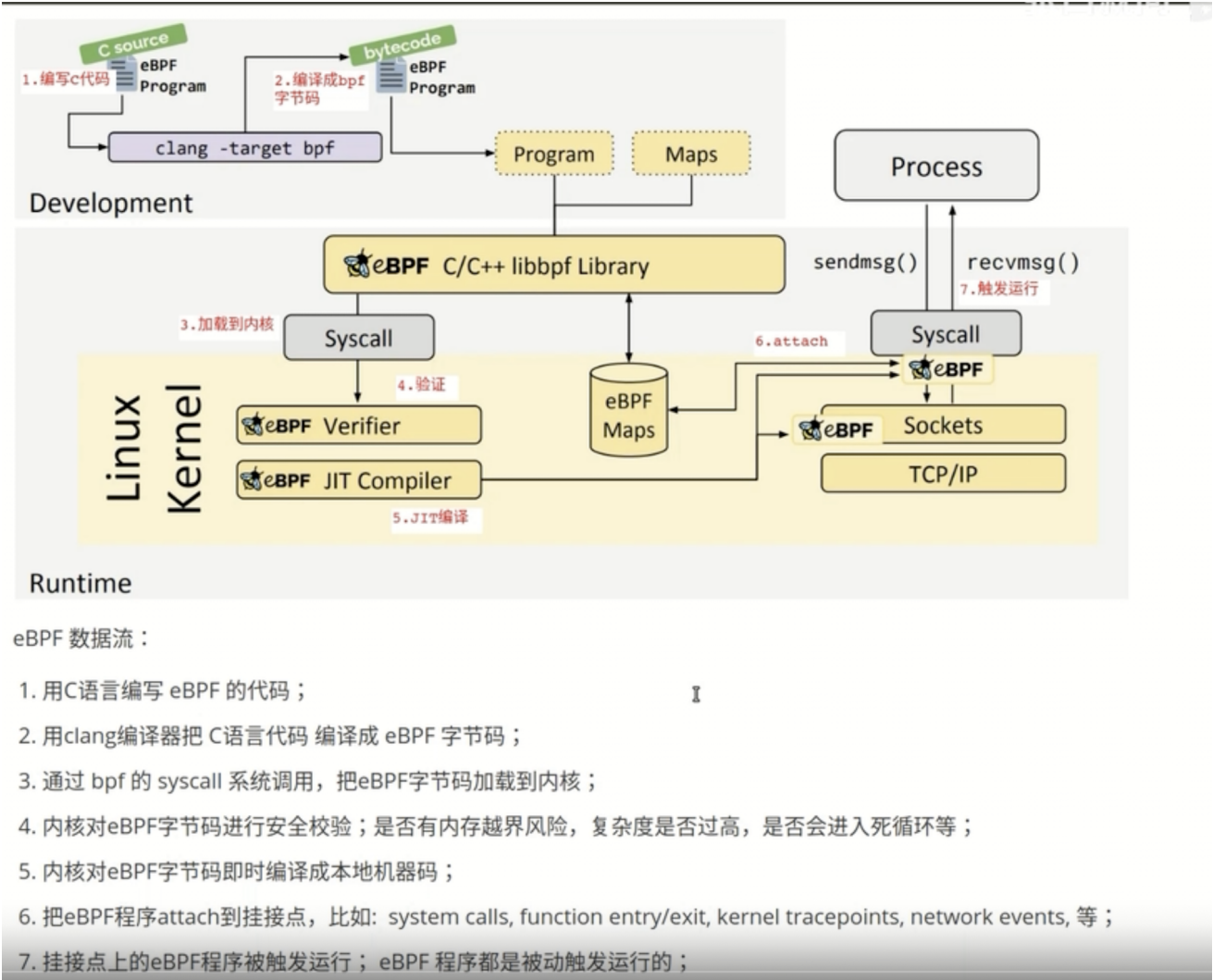
- **BCC Python API**: 通过 Python 提供了访问 eBPF 的接口，开发者可以使用 Python 脚本来编写、加载和管理 eBPF 程序。
- **bpfcc 工具**: 包括一些内核跟踪和性能分析工具，用于实时监控内核行为。
- **bpftool**: 基于 eBPF 的动态跟踪工具，允许开发者使用类 `awk` 的脚本语言快速编写 eBPF 程序，用于分析和调试。
- **内核跟踪工具**: 例如，`trace` 工具可以追踪系统调用、函数调用和硬件事件，帮助开发者理解内核行为。

BCC 如何帮助开发者

- **简化 eBPF 编程**：通过为开发者提供高级抽象，减少了直接编写低级内核代码的需要。你可以通过 BCC 使用更熟悉的语言（如 Python）来编写 eBPF 程序。
- **快速实现监控工具**：BCC 提供了一些现成的工具和脚本，可以帮助开发者快速实现监控功能。你可以用它来监控系统调用、网络事件、文件操作等，而无需从头开始编写代码。
- **内核与用户空间的桥梁**：开发者不必直接与内核空间交互，BCC 自动处理 eBPF 程序的加载和数据交换。

总结：

BCC 在 eBPF 开发中起到了非常重要的作用，它简化了开发过程，使得编写、调试、监控和分析 eBPF 程序变得更加容易。通过提供高层的 API 和现成的工具，BCC 降低了开发门槛，并帮助开发者更好地利用 eBPF 技术进行性能优化、系统监控和故障诊断。如果你打算开发 eBPF 程序，BCC 是一个非常有用的工具集，特别适合快速上手和高效开发。



eBPF 数据流：

1. 用C语言编写 eBPF 的代码；
2. 用clang编译器把 C语言代码 编译成 eBPF 字节码；
3. 通过 bpf 的 syscall 系统调用，把eBPF字节码加载到内核；
4. 内核对eBPF字节码进行安全校验；是否有内存越界风险，复杂度是否过高，是否会进入死循环等；
5. 内核对eBPF字节码即时编译成本地机器码；
6. 把eBPF程序attach到挂接点，比如： system calls, function entry/exit, kernel tracepoints, network events, 等；
7. 挂接点上的eBPF程序被触发运行； eBPF 程序都是被动触发运行的；